Raspberry Pi Pico as a CMSIS-DAP Debug Probe for ARM Cortex-M Development

> **A complete guide to converting a $4 Raspberry Pi Pico into a professional SWD debugger using the official Debug Probe firmware — validated on nRF52840 and STM32F411 Discovery.**

Table of Contents

Overview

Hardware debugging is the backbone of embedded software development. Without a proper debug probe, you are limited to blinking LEDs and UART print statements — which is slow and painful for anything beyond the simplest firmware.

Commercial debug probes such as the **SEGGER J-Link** and **Keil ULINK** are excellent tools, but they carry a significant cost. For hobbyists, students, or engineers working on personal projects, this cost is often unjustifiable.

The **Raspberry Pi Pico** solves this problem. By loading the official Raspberry Pi Debug Probe firmware, the Pico becomes a fully functional **CMSIS-DAP compatible SWD debugger and programmer** — capable of everything a commercial probe can do for most ARM Cortex-M development scenarios.

This guide covers the entire workflow from scratch:

- Building and flashing the debug probe firmware onto the Pico
- Wiring the Pico to your target microcontroller
- Configuring and using OpenOCD
- Flashing firmware, erasing flash, and debugging with GDB
- Integrating everything into Visual Studio Code

Why This Matters

Feature	J-Link EDU	Raspberry Pi Pico Debug Probe
Price	~$18 USD (EDU license)	~$4 USD
Protocol	SWD / JTAG	SWD
OpenOCD Support	Yes	Yes
GDB Support	Yes	Yes
VS Code Support	Yes	Yes
Commercial Use	Restricted (EDU)	Unrestricted

For SWD-based ARM Cortex-M development, the Pico debug probe covers nearly everything you need at a fraction of the cost.

Features

- **CMSIS-DAP v1/v2 compatible** — works with any tool that supports the CMSIS-DAP standard
- **SWD (Serial Wire Debug)** — the standard ARM debug interface
- **OpenOCD support** — industry-standard open source on-chip debugger

- **GDB integration** — full source-level debugging with breakpoints, watchpoints, memory inspection
- **Visual Studio Code support** — via Cortex-Debug extension
- **Firmware flashing** — program ELF, HEX, and BIN files directly
- **Flash mass erase** — erase entire flash contents
- **Breakpoint debugging** — halt execution and step through code line by line
- **Memory and register inspection** — read/write RAM, flash, and peripheral registers at runtime
- **Low cost** — total hardware cost under $5 USD

Supported Target Devices

Any ARM Cortex-M microcontroller that exposes an SWD interface can be used as a target.

Examples:

Manufacturer	Series
Nordic Semiconductor	nRF51, nRF52 (nRF52832, nRF52840, etc.)
STMicroelectronics	STM32F0/F1/F3/F4/F7/G0/G4/H7/L0/L4/WB/WL
Raspberry Pi	RP2040
NXP	LPC Series
Microchip	SAMD Series
Any other vendor	Any Cortex-M exposing SWCLK, SWDIO, GND

Validated in this guide:

- **Pro Micro nRF52840** (Nordic Semiconductor nRF52840)
- **STM32F411 Discovery Board** (STMicroelectronics STM32F411)

The overall workflow is identical across targets — only the OpenOCD target configuration file changes.

What You Need

Hardware:

- Raspberry Pi Pico (RP2040) — this becomes your debug probe
- Target ARM Cortex-M board — the device you want to flash and debug
- 3x jumper wires (minimum) — to connect SWCLK, SWDIO, and GND

Software:

- Git
- CMake (3.13 or later)
- ARM GCC Toolchain (`arm-none-eabi-gcc`)
- Raspberry Pi Pico SDK
- OpenOCD (with CMSIS-DAP support)
- Visual Studio Code (optional, for IDE integration)
- Cortex-Debug VS Code Extension (optional)

Part 1 — Building the Debug Probe Firmware

The debug probe firmware is the official Raspberry Pi project that transforms the Pico into a CMSIS-DAP interface. You need to compile it from source.

Step 1.1 — Clone the Repository

bash

```
# Clone the official debugprobe repository from Raspberry Pi
git clone https://github.com/raspberrypi/debugprobe.git

# Enter the project directory
cd debugprobe
```

Step 1.2 — Create the Build Directory

CMake out-of-source builds keep generated files separate from source files.
This is standard practice and makes cleaning up easy.

bash

```
mkdir build
cd build
```

Step 1.3 — Configure with CMake

The `DEBUG_ON_PICO=ON` flag configures the build specifically for the Raspberry Pi Pico (RP2040). Without this flag, the build targets the official Raspberry Pi Debug Probe hardware accessory instead.

```bash
cmake -DDEBUG_ON_PICO=ON ..
```

> **Note:** Before running this, make sure the `PICO_SDK_PATH` environment variable is set and points to your Pico SDK installation directory. If not set, CMake will attempt to fetch the SDK automatically via git submodule.

Step 1.4 — Build the Firmware

Use all available CPU cores to speed up compilation:

```bash
make -j$(nproc)
```

On macOS, use `make -j$(sysctl -n hw.logicalcpu)` instead.

Step 1.5 — Locate the Output File

After a successful build, the output file you need is:

```
build/debugprobe_on_pico.uf2
```

The `.uf2` file is the UF2 (USB Flashing Format) image — a format designed specifically for drag-and-drop flashing on RP2040 devices.

Part 2 — Flashing the Firmware onto the Pico

This step loads the debug probe firmware onto your Raspberry Pi Pico using the built-in USB bootloader.

Step 2.1 — Enter Bootloader Mode

1. **Hold down the BOOTSEL button** on the Pico (the white button on the top face of the

board)
2. While holding BOOTSEL, **connect the Pico to your PC via USB**
3. **Release the BOOTSEL button** after the USB cable is fully connected

The Pico will appear on your PC as a USB mass storage device named `RPI-RP2` — exactly like a USB flash drive.

Step 2.2 — Copy the UF2 File

Drag and drop (or copy) the firmware file onto the mounted drive:

```bash
# Linux example — adjust the mount path as needed
cp build/debugprobe_on_pico.uf2 /media/$USER/RPI-RP2/

# macOS example
cp build/debugprobe_on_pico.uf2 /Volumes/RPI-RP2/
```

The Pico will:

1. Automatically detect the UF2 file
2. Flash the firmware to its internal flash memory
3. Reboot itself
4. Re-enumerate on USB — this time as a **CMSIS-DAP debug probe**, not a storage device

At this point, the Pico is no longer a general-purpose microcontroller. It is now a dedicated debug probe.

Part 3 — Hardware Wiring

Connect the Pico (debug probe) to your target ARM Cortex-M board using the SWD interface.

Debug Probe GPIO Pin Assignments

The debugprobe firmware uses specific GPIO pins for the SWD signals:

Raspberry Pi Pico Pin	GPIO	SWD Function
Pin 4	GP2	SWCLK (Serial Wire Clock)
Pin 5	GP3	SWDIO (Serial Wire Data I/O)
Any GND pin	GND	Ground reference

Wiring Table

Connect the Pico to your target as follows:

Pico (Debug Probe)	Target MCU
GP2	SWCLK
GP3	SWDIO
GND	GND

Optional — powering the target from the Pico:

Pico	Target
3V3 (Pin 36)	VCC (3.3V rail)

> **Production recommendation:** Power the target board independently from its own power source or USB connection. Only connect SWCLK, SWDIO, and GND between the Pico and the target. This avoids ground loops, power sequencing issues, and current overload on the Pico's 3.3V regulator.

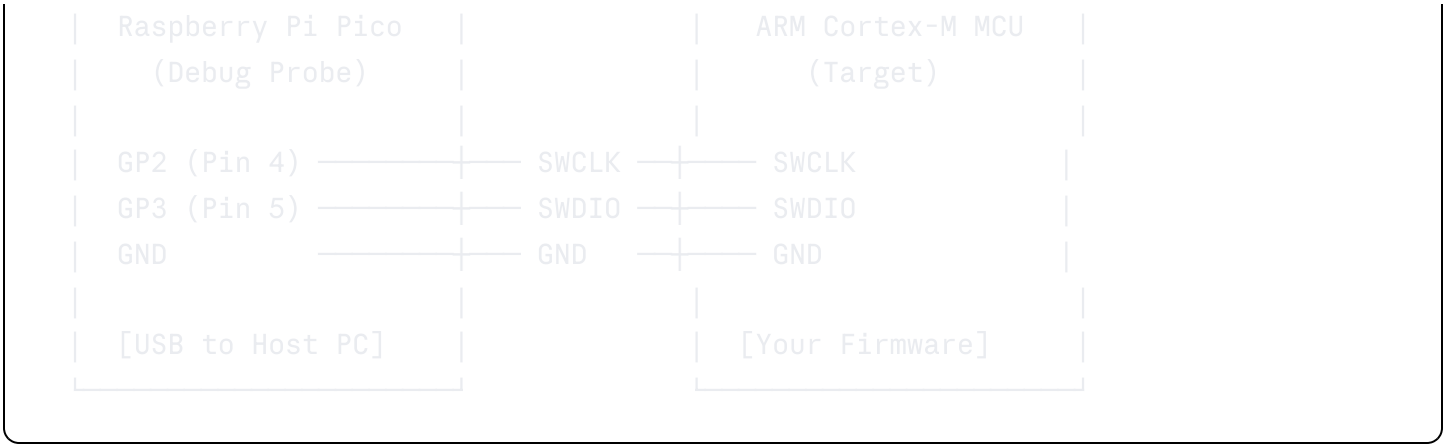
Connection Diagram

The images below show the physical wiring between the Raspberry Pi Pico debug probe and the nRF52840 target:

> 📷 **[Connection diagram image — add your wiring photo here]**

> 📷 **[Close-up of Pico GP2/GP3 connections — add your photo here]**

Wiring Schematic (ASCII)



Part 4 — Verifying Probe Detection

Before attempting any debug operations, confirm that the operating system has detected the Pico as a CMSIS-DAP device.

Linux

```bash
lsusb
```

Look for a line containing `CMSIS-DAP` in the output. Example:

```
Bus 001 Device 005: ID 2e8a:000c Raspberry Pi CMSIS-DAP v2 Interface
```

For more detail:

```bash
# Show USB device details
lsusb -v | grep -A 10 CMSIS-DAP
```

macOS

```bash
system_profiler SPUSBDataType | grep -A 5 "CMSIS"
```

Windows

Open **Device Manager** → look under **Universal Serial Bus devices** for `CMSIS-DAP`.

If the device is not detected, check:

- The USB cable (try a different cable — some cables are charge-only)
- Whether the firmware was flashed correctly (repeat Part 2)
- USB port (try a different port or hub)

Part 5 — OpenOCD Integration

OpenOCD (Open On-Chip Debugger) is the software bridge between the debug probe and your target MCU. It speaks CMSIS-DAP to the Pico and exposes a GDB server that your debugger connects to.

Installing OpenOCD

Ubuntu / Debian:

```bash
sudo apt-get install openocd
```

macOS (Homebrew):

```bash
brew install openocd
```

Windows:

Download from https://openocd.org/pages/getting-openocd.html or use the xPack release.

> **Important:** Make sure your OpenOCD version includes CMSIS-DAP support. Version 0.11.0 or later is recommended.

Basic OpenOCD Launch Command

The general form for launching OpenOCD with the Pico debug probe is:

```bash
openocd \
  -f interface/cmsis-dap.cfg \   # Selects the CMSIS-DAP interface (our Pico probe)
  -f target/<your_target>.cfg    # Selects the target MCU configuration
```

Target-Specific Commands

Replace `<your_target>.cfg` with the correct file for your MCU:

Nordic nRF52 series (nRF52840, nRF52832, etc.):

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/nrf52.cfg
```

STM32F4 series (STM32F411, STM32F407, STM32F446, etc.):

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/stm32f4x.cfg
```

Raspberry Pi RP2040:

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/rp2040.cfg
```

STM32F1 series (STM32F103, Blue Pill, etc.):

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/stm32f1x.cfg
```

STM32L4 series:

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/stm32l4x.cfg
```

Expected Output

When OpenOCD connects successfully to the target, you should see output similar to:

```
Info : CMSIS-DAP: SWD  Supported
Info : CMSIS-DAP: FW Version = 2.0.0
Info : CMSIS-DAP: Interface Initialised (SWD)
Info : SWCLK/TCK = 1 SWDIO/TMS = 1 TDI = 0 TDO = 0 nTRST = 0 nRESET = 1
Info : CMSIS-DAP: Interface ready
Info : clock speed 1000 kHz
Info : SWD DPIDR 0x2ba01477         ← ARM CoreSight Debug Port ID
Info : nrf52.cpu: hardware has 6 breakpoints, 4 watchpoints
Info : starting gdb server for nrf52.cpu on 3333
Info : Listening on port 3333 for gdb connections
```

Once OpenOCD is running, it exposes:

- **Port 3333** — GDB server (for debugging)
- **Port 4444** — Telnet command interface (for scripting)
- **Port 6666** — TCL interface

Leave this terminal open while debugging.

Part 6 — Flash Programming

Firmware images are programmed through OpenOCD using the `program` command. This works with ELF, Intel HEX, and raw binary files.

Flashing nRF52840

bash

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/nrf52.cfg \
  -c "init; reset halt; program firmware.hex verify reset exit"
```

Command breakdown:

Command	Description
`init`	Initialize the debug probe and connect to the target
`reset halt`	Reset the target MCU and immediately halt execution at reset vector
`program firmware.hex`	Erase the affected sectors and write the firmware image
`verify`	Read back the written data and verify it matches the input file
`reset`	Release reset and allow the target to run the new firmware
`exit`	Close OpenOCD after the operation completes

Flashing STM32F411

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/stm32f4x.cfg \
  -c "init; reset halt; program firmware.elf verify reset exit"
```

> **ELF vs HEX vs BIN:**
> - **ELF** — includes symbol information; preferred for debugging sessions
> - **HEX** — Intel HEX format; commonly produced by Keil, IAR, and STM32CubeIDE
> - **BIN** — raw binary; requires specifying the base address: `program firmware.bin 0x08000000 verify reset exit`

Flashing RP2040

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/rp2040.cfg \
  -c "init; reset halt; program firmware.elf verify reset exit"
```

Part 7 — Mass Erase

A mass erase wipes the entire flash memory of the target MCU. This is useful when:

- The existing firmware has set read-protection that blocks normal access
- You need a clean slate before flashing new firmware
- You are clearing softdevices or bootloaders from nRF52 targets

Mass Erase — nRF52840

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/nrf52.cfg \
  -c "init; reset halt; nrf5 mass_erase; reset; exit"
```

> **Note for nRF52840:** If the chip has APPROTECT (Access Port Protection) enabled, a mass erase will also disable the protection, restoring full debug access. This is the standard recovery procedure for locked nRF52 devices.

Mass Erase — STM32F4

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/stm32f4x.cfg \
  -c "init; reset halt; flash erase_address 0x08000000 0x80000; reset; exit"
```

Or using the `flash erase_sector` command for fine-grained control.

Part 8 — Visual Studio Code Integration

VS Code with the Cortex-Debug extension provides a powerful IDE-level debugging experience. The following configuration integrates OpenOCD and the Pico debug probe directly into VS Code's build and debug system.

tasks.json

Add these tasks to `.vscode/tasks.json` in your project. They allow you to flash firmware and erase flash directly from VS Code using **Terminal → Run Task**.

```json
```

```jsonc
{
    "version": "2.0.0",
    "tasks": [

        // ── Flash and Run ──────────────────────────────────────────────
        // Programs the target MCU with the compiled firmware, verifies the
        // write, resets the target, and exits OpenOCD.
        // Adjust the firmware path to match your build output.
        {
            "label": "Flash and Run — nRF52840",
            "type": "shell",
            "command": "openocd",
            "args": [
                "-f", "interface/cmsis-dap.cfg",
                "-f", "target/nrf52.cfg",
                "-c", "init; reset halt; program ${workspaceFolder}/build/firmware.h
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "presentation": {
                "reveal": "always",
                "panel": "shared"
            },
            "problemMatcher": []
        },

        // ── Flash and Run — STM32F411 ──────────────────────────────────
        {
            "label": "Flash and Run — STM32F411",
            "type": "shell",
            "command": "openocd",
            "args": [
                "-f", "interface/cmsis-dap.cfg",
                "-f", "target/stm32f4x.cfg",
                "-c", "init; reset halt; program ${workspaceFolder}/build/firmware.e
            ],
            "group": "build",
            "presentation": {
                "reveal": "always",
                "panel": "shared"
            },
```

```
            "problemMatcher": []
        },

        // ── Mass Erase ──────────────────────────────────────────────
        // Erases all flash contents on the nRF52840.
        // Also disables APPROTECT (readback protection) if it was enabled.
        {
            "label": "Mass Erase — nRF52840",
            "type": "shell",
            "command": "openocd",
            "args": [
                "-f", "interface/cmsis-dap.cfg",
                "-f", "target/nrf52.cfg",
                "-c", "init; reset halt; nrf5 mass_erase; reset; exit"
            ],
            "group": "build",
            "presentation": {
                "reveal": "always",
                "panel": "shared"
            },
            "problemMatcher": []
        }

    ]
}
```

launch.json

Add this to `.vscode/launch.json` for full GDB debugging support via Cortex-Debug:

json

```jsonc
{
    "version": "0.2.0",
    "configurations": [

        // ── Debug — nRF52840 ──────────────────────────────────────────
        // Launches OpenOCD in the background, connects GDB, flashes the
        // firmware, and halts at main() ready for breakpoint debugging.
        {
            "name": "Debug — nRF52840 (CMSIS-DAP)",
            "type": "cortex-debug",
            "request": "launch",
            "servertype": "openocd",
            "cwd": "${workspaceFolder}",

            // Path to your compiled ELF with debug symbols
            "executable": "${workspaceFolder}/build/firmware.elf",

            // OpenOCD configuration files — must match your target
            "configFiles": [
                "interface/cmsis-dap.cfg",
                "target/nrf52.cfg"
            ],

            // Halt at main() on launch
            "runToEntryPoint": "main",

            // SVD file enables peripheral register view in VS Code
            // Download from: https://github.com/posborne/cmsis-svd
            "svdFile": "${workspaceFolder}/nRF52840.svd"
        },

        // ── Debug — STM32F411 ─────────────────────────────────────────
        {
            "name": "Debug — STM32F411 (CMSIS-DAP)",
            "type": "cortex-debug",
            "request": "launch",
            "servertype": "openocd",
            "cwd": "${workspaceFolder}",
            "executable": "${workspaceFolder}/build/firmware.elf",
            "configFiles": [
                "interface/cmsis-dap.cfg",
                "target/stm32f4x.cfg"
            ],
```

```
            "runToEntryPoint": "main",
            "svdFile": "${workspaceFolder}/STM32F411.svd"
        }

    ]
}
```

> **Cortex-Debug extension:** Install it from the VS Code marketplace — search for `marus25.cortex-debug`. It provides the `cortex-debug` debug type used above.

Part 9 — GDB Debugging Workflow

Starting a Debug Session Manually

Terminal 1 — Launch OpenOCD (leave running):

bash

```
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/nrf52.cfg
```

Terminal 2 — Launch GDB:

bash

```
# For nRF52840
arm-none-eabi-gdb build/firmware.elf

# Inside GDB:
(gdb) target extended-remote :3333   # Connect to OpenOCD GDB server
(gdb) monitor reset halt             # Reset and halt the target
(gdb) load                           # Flash the firmware
(gdb) monitor reset halt             # Reset again, halt at reset vector
(gdb) break main                     # Set breakpoint at main()
(gdb) continue                       # Run until breakpoint
```

Useful GDB Commands

bash

```
# Execution control
(gdb) continue          # Run until next breakpoint
(gdb) step              # Step into next line (enters function calls)
(gdb) next              # Step over next line (does not enter functions)
(gdb) finish            # Run until current function returns
(gdb) interrupt         # Halt a running target

# Breakpoints
(gdb) break main            # Breakpoint at function entry
(gdb) break file.c:42       # Breakpoint at specific file and line
(gdb) break *0x00001234     # Breakpoint at specific address
(gdb) info breakpoints      # List all breakpoints
(gdb) delete 1              # Delete breakpoint number 1

# Memory inspection
(gdb) x/10xw 0x20000000    # Print 10 words at address (hex)
(gdb) x/s 0x20000000       # Print string at address
(gdb) info registers        # Dump all CPU registers

# Variable inspection
(gdb) print my_variable         # Print variable value
(gdb) print &my_variable        # Print variable address
(gdb) display my_variable       # Auto-display on each step
(gdb) watch my_variable         # Break when variable changes (watchpoint)

# Stack
(gdb) backtrace             # Show call stack
(gdb) frame 2               # Switch to stack frame 2

# OpenOCD commands via GDB
(gdb) monitor reset halt        # Reset and halt target
(gdb) monitor flash erase_sector 0 0 0   # Erase sector
(gdb) monitor mdw 0x20000000    # Read word from memory via OpenOCD
```

Development Cycle

A typical iteration looks like this:

```
1. Edit source code
2. Build firmware (make / cmake --build)
3. Flash via OpenOCD or VS Code task
4. Launch GDB or press F5 in VS Code
5. Set breakpoints
6. Step through code, inspect variables and registers
7. Identify bug
8. Fix and repeat
```

Validated Hardware

This setup has been tested and confirmed working on the following hardware combinations:

Debug Probe	Target Board	MCU	Status
Raspberry Pi Pico	Pro Micro nRF52840	Nordic nRF52840	✅ Verified
Raspberry Pi Pico	STM32F411 Discovery	STM32F411VET6	✅ Verified

Validated operations:

Operation	nRF52840	STM32F411
OpenOCD CMSIS-DAP connection	✅	✅
Firmware flashing (HEX/ELF)	✅	✅
Flash verification	✅	✅
Mass erase	✅	✅
GDB breakpoint debugging	✅	✅
Memory and register inspection	✅	✅
VS Code Cortex-Debug integration	✅	✅

> 📷 **[Hardware setup photo with nRF52840 — add your image here]**

> 📷 **[OpenOCD successful connection terminal screenshot — add your image here]**

OpenOCD says "Error connecting DP: cannot read IDR"

- Check wiring — SWCLK and SWDIO may be swapped
- Verify the target has power
- Try a lower SWD clock speed: add `-c "adapter speed 500"` before `-f target/...`
- Make sure GND is connected between Pico and target

"No CMSIS-DAP device found" from OpenOCD

- Confirm the Pico has the debugprobe firmware (check with `lsusb`)
- Try a different USB cable or port
- On Linux, you may need udev rules:

```bash
# Create a udev rule for CMSIS-DAP
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="2e8a", ATTR{idProduct}=="000c", MODE="0666"
  | sudo tee /etc/udev/rules.d/60-cmsis-dap.rules
sudo udevadm control --reload-rules
```

Then replug the Pico.

nRF52840 is locked (APPROTECT enabled)

Perform a mass erase to recover:

```bash
openocd \
  -f interface/cmsis-dap.cfg \
  -f target/nrf52.cfg \
  -c "init; reset halt; nrf5 mass_erase; reset; exit"
```

GDB cannot connect to port 3333

- Make sure OpenOCD is running and has connected successfully to the target
- Check for firewall rules blocking localhost port 3333
- Check that no other OpenOCD instance is already running: `pkill openocd`

- Try halting the target before programming: ensure `reset halt` appears before `program`
- Some STM32 devices require unlocking the flash: add `stm32f4x unlock 0` before programming

---

References

- Raspberry Pi Debug Probe — Official Repository
- Raspberry Pi Pico SDK
- OpenOCD Documentation
- CMSIS-DAP Standard — ARM
- Cortex-Debug VS Code Extension
- Nordic nRF52840 Product Page
- STM32F411 Discovery Board

License

Tested on Ubuntu 22.04 LTS. Workflow is compatible with macOS and Windows (with WSL or native toolchains).